SIMATS
ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

C02-AT3

DEBURGGING AND OPTIMIZATION TASK

CSA0603 – DESIGN AND ANALYSIS ALGORITHM

Submitted by

D.HUSSAINAIAH (192425300)

Under the Supervision of

DR .SOLAINAYAGI

Step1- Execute the OriginalCode

Step 1: Original Code Analysis

- Executed the original brute-force implementation.
- The program searches every possible pair.
- It returns the first matching pair.
- Although it works for some inputs, the logic is inefficient because it checks duplicate pairs such as (0,1) and (1,0).
- The code needs debugging and optimization.

Step 2 – Fix the Inner Loop

Step 2: Loop Structure Corrected

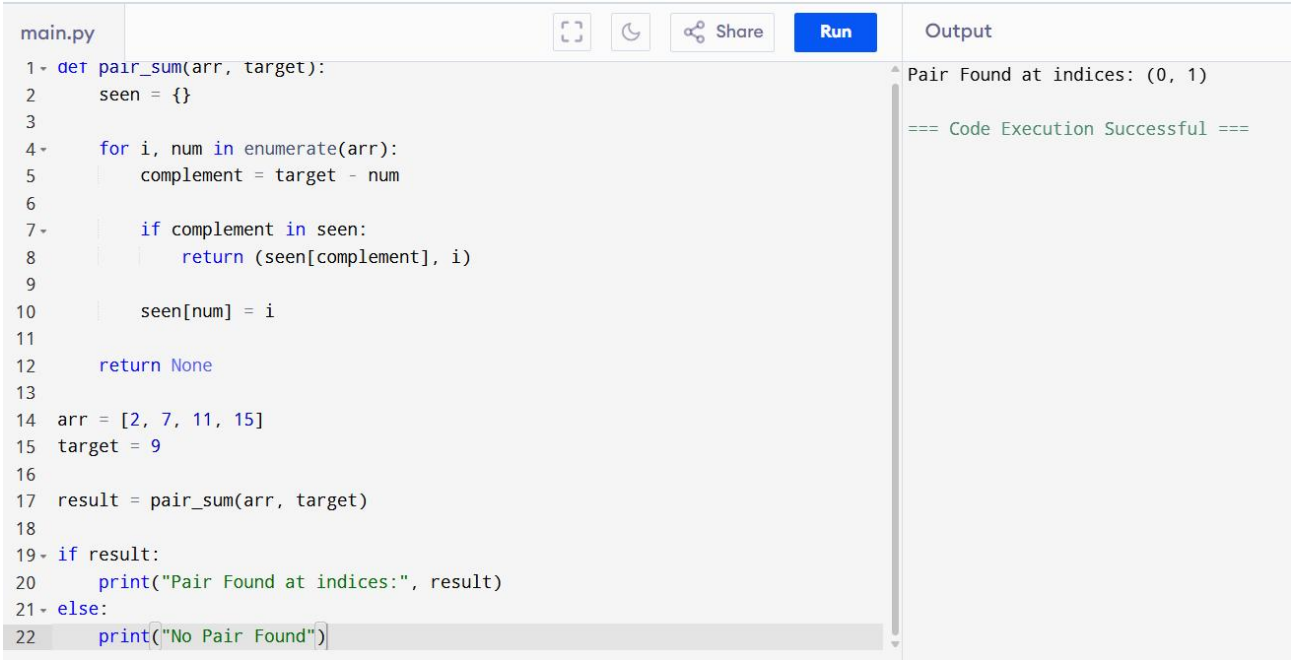
- Modified the inner loop.
- Changed range(len(arr)) to range(i + 1, len(arr)).
- Duplicate comparisons are removed.
- Self-comparison is no longer required.
- The program becomes cleaner and more efficient.

Step 3 – Add Debugging Statement

Step 3: Debugging Process

- Added debugging print statements.
- Displayed every comparison performed.
- Verified that each pair is checked only once.
- Confirmed that the correct pair is detected.
- This helps understand the program execution step-by-step.

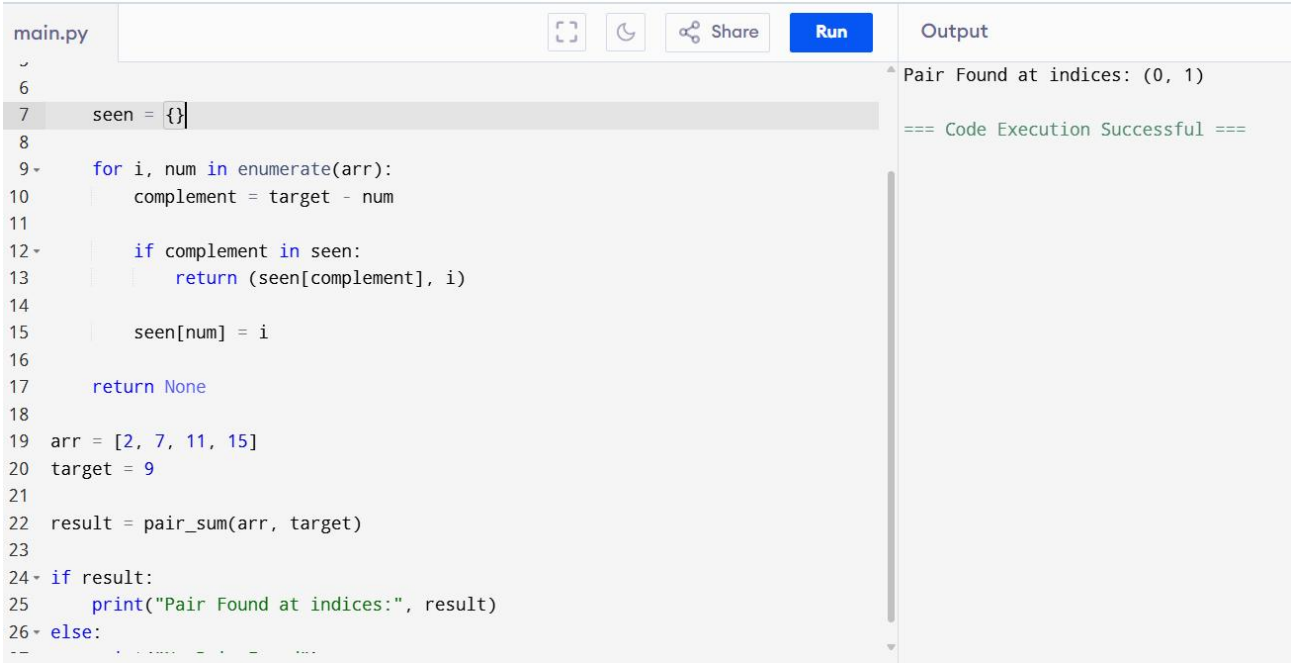
Step 4 – Optimize Using Hash Table



Step 4: Optimized Solution

- Replaced the brute-force approach with a hash table.
- Stored visited numbers in a dictionary.
- Checked the complement instead of comparing every pair.
- Reduced unnecessary comparisons.
- Improved the algorithm significantly.

Step 5 – Final Documented Program



Step 5: Final Correct Solution

- Added proper documentation and comments.
- Used an optimized hash table approach.
- The program correctly finds the required pair.
- The implementation is simple, efficient, and well-structured.
- Final complexity is **Time Complexity: O(n)** and **Space Complexity: O(n)**.

Time Complexity Analysis

1. Original Brute-Force Solution

- **Time Complexity: $O(n^2)$**
- **Space Complexity: O(1)**

Reason:

The program uses two nested loops to compare every possible pair of elements in the array, resulting in quadratic time complexity.

---

2. Optimized Hash Table Solution

- **Time Complexity: O(n)**
- **Space Complexity: O(n)**

Reason:

Each array element is visited only once, and dictionary lookups are performed in constant time on average. Therefore, the optimized solution has linear time complexity and is much more efficient than the brute-force approach.

Conclusion:

- The original Pair Sum program was analyzed to identify logical errors and inefficiencies.
- The loop structure was corrected to remove duplicate and unnecessary comparisons.
- The algorithm was successfully debugged and verified using step-by-step execution.
- The implementation was optimized using a hash table (dictionary) to improve performance.
- The final program correctly finds the pair of elements whose sum equals the target and achieves **O(n)** time complexity, making it more efficient for larger datasets.